

BugFlow Platform Documentation

Comprehensive Feature Catalog & Technical Implementation Architecture

Version: 2.4.0 | Framework: FastAPI + React + PostgreSQL | Status: Production Ready

1. System Architecture & Technology Stack

BugFlow is an AI-powered defect intelligence and bug tracking platform designed for modern engineering teams. It couples FastAPI's high-throughput asynchronous backend with React 18, dense semantic vector embeddings, and ReportLab PDF compilation to streamline the entire software QA lifecycle.

Component Layer	Technologies Used	Key Responsibilities
Frontend Client	React 18, Vite, Vanilla CSS, Lucide React	Interactive UI, real-time forms, responsive Kanban, theme toggle
Backend API	FastAPI, Pydantic v2, Python 3.14	Async REST endpoints, auth, validation, activity logging
Database	PostgreSQL, SQLAlchemy 2.0 ORM	Relational models: users, issues, projects, sprints, comments
AI Engine	Dense Concept Vectors, OpenAI Embeddings, GPT-4o-mini	Semantic search, classification, duplicate check, resolution copilot
PDF Engine	ReportLab Document Engine	Dynamic generation of defect investigation & project summary PDFs
Security & Auth	Passlib (bcrypt), PyJWT, RBAC	JWT tokens, role guards (Admin, PM, Developer, QA, Reporter)

2. Intelligent Defect Classification

How It Works: When a QA tester enters a defect description, the system automatically analyzes the domain context and suggests **Category** (e.g. *Payment, Auth, Performance*), **Module** (e.g. *Checkout / Gateway*), **Defect Type** (e.g. *Functional Defect, Security Vulnerability*), and **Suggested Severity & Priority**. The tester can accept all suggestions with 1 click or adjust them individually.

Technical Implementation:

- **Database:** Added category, module, and defect_type columns to Issue model (models.py).
- **AI Core:** classify_defect() in app/ai.py runs structured classification with heuristic domain fallbacks.
- **API Endpoint:** POST /api/ai/classify-defect in app/routers/ai.py.
- **Frontend:** IssueForm.jsx renders an interactive suggestion card with 1-click apply and datalists.

3. Line-by-Line Structured Description Formatting

How It Works: AI-generated and expanded bug descriptions are structured cleanly line-by-line with standard section headers (Summary:, Steps to Reproduce:, Expected Result:, Actual Result:, Impact:) with numbered items and bullet

points.

Technical Implementation:

- **Prompting:** SYSTEM_PROMPT in ai.py strictly specifies line-by-line vertical format.
- **CSS Styling:** .markdown-content in index.css applies white-space: pre-wrap; line-height: 1.7; ensuring clean line separation.

4. Intelligent Severity & Priority Assistance

How It Works: Analyzes defect descriptions to evaluate technical and user impact (e.g., 'All users are unable to complete payment' → **Critical Severity**). Displays an explicit disclaimer: 'AI suggestion provided. The final decision remains with the authorized user.'

Technical Implementation:

- **AI Evaluator:** predict_severity() in ai.py scores system impact keywords.
- **API Endpoint:** POST /api/ai/predict-severity returning SeverityPredictResponse.

5. Similar Defect Detection & Duplicate Prevention

How It Works: When typing a new bug, the system automatically checks existing defects in the project. If a potential duplicate exists (e.g., New: 'Application crashes when submitting payment' vs Existing: 'Payment submission causes application crash'), it warns the tester:

■ Similar Defect Found: DEF-102 (97% Match) | DEF-156 (68% Match)

Technical Implementation:

- **Similarity Engine:** calculate_defect_similarity() in ai.py applies domain word-stemming and token overlap scoring.
- **UI Component:** IssueForm.jsx displays an alert banner listing matched keys before form submission.

6. Semantic Vector Search Engine

How It Works: Enables searching defects by conceptual meaning rather than exact keywords. For example, searching 'Payment fails after clicking submit' finds 'Transaction crashes during checkout' with a **97.2% Semantic Match**.

Technical Implementation:

- **Dense Vector Math:** build_semantic_vector() and cosine_similarity() in ai.py map queries into concept cluster spaces.
- **API Endpoint:** POST /api/ai/semantic-search in routers/ai.py.
- **Frontend UI:** Dashboard.jsx integrates a ■ **Semantic Search** toggle with match percentage badges.

7. Signature Feature: Resolution Assistance Copilot

How It Works: When a developer opens any defect, the system generates comprehensive technical guidance:

1. **Possible Investigation Areas:** Interactive checklist (API responses, null checks, error handlers, server logs).
2. **Similar Defects:** Historical defect IDs (DEF-xxx) with similarity metrics.
3. **Previous Resolution:** How similar historical defects were resolved.
4. **Possible Resolution:** Specific recommended technical fix with 'Copy Fix' and 'Post to Comments' buttons.

Technical Implementation:

- **Backend Logic:** generate_resolution_assistance() in app/ai.py.

- **API Route:** GET /api/issues/{id}/resolution-assistance in routers/issues.py.
- **UI Modal:** BugDetailModal.jsx dedicated '■ Resolution Assistance' tab.
- **PDF Integration:** Embedded into defect PDF reports via pdf_service.py.

8. Code Doctor (Pure Direct Syntax & Semantic Repair)

How It Works: When faulty code is submitted (e.g. `print("hell")`), Code Doctor returns the direct fixed code (`print("hell")`) without wrapping in unnecessary `try...catch` blocks. If code is already valid, it confirms `is_correct: true`.

Technical Implementation:

- **AST Validation:** `fix_code_snippet()` in `ai.py` runs Python AST checks before repair.
- **Frontend Editor:** `CodeDoctor.jsx` side-by-side comparison editor with diff highlights and 1-click copy.

9. 1-Click PDF Report Export (Issues & Projects)

How It Works: Generates branded, structured PDF documents on demand:

- **Defect Report PDF:** Metadata grid, line-by-line description, reproduction steps, resolution assistance, and discussion history.
- **Project Summary PDF:** Project metrics, severity distributions, and full defect inventory table.

Technical Implementation:

- **ReportLab Engine:** `pdf_service.py` `generate_issue_pdf()` and `generate_project_summary_pdf()`.
- **Streaming Endpoints:** GET /api/issues/{id}/pdf and GET /api/projects/{id}/pdf.
- **Client Trigger:** `api.js` binary blob stream download in `BugDetailModal.jsx` and `Dashboard.jsx`.

10. Defect Editing & Lifecycle Management

How It Works: Enables complete editing of existing bugs from:

- **Bug Detail Modal:** Header '■ Edit Bug' toggle to edit fields in-place.
- **Table View:** Dedicated '■ Edit' button on every defect row.
- **Kanban Board:** Quick '■' edit icon on each card.

Technical Implementation:

- **API Endpoint:** PUT /api/issues/{id} in routers/issues.py.
- **UI Component:** `IssueForm.jsx` handles both creation and editing via initial prop syncing.

11. User Profile & Account Settings Management

How It Works: Users can click their user profile card in the bottom-left sidebar to open the **User Profile Modal** to:

- Update **Username** and **Email Address**.
- Change **Security Password** (with current password verification).
- View assigned role badge and join date with real-time feedback.

Technical Implementation:

- **API Route:** PUT /auth/profile in routers/auth.py.
- **Frontend Component:** `ProfileModal.jsx` with immediate `AuthContext` state sync.

12. Project Management, Sprints & Interactive Kanban

How It Works: Complete multi-project environment with sprint planning, sprint risk prediction, 5-column drag/click Kanban workflow (Open → In Progress → In Review → Resolved → Closed), and visual analytics charts.

Technical Implementation:

- **Routers:** routers/projects.py, routers/sprints.py, routers/dashboard.py.
- **Components:** SprintManager.jsx, Dashboard.jsx SVG chart renderers.

13. Key API Endpoints & Routes Matrix

Method & Route	Controller	Functionality
POST /auth/login	auth.py	Authenticates user and returns JWT token
PUT /auth/profile	auth.py	Updates username, email, and password
GET /api/issues	issues.py	Lists project defects with filtering & search
PUT /api/issues/{id}	issues.py	Updates all defect fields & logs activity
GET /api/issues/{id}/pdf	issues.py	Streams individual defect PDF report
GET /api/projects/{id}/pdf	projects.py	Streams project summary PDF report
POST /api/ai/classify-defect	ai.py	Suggests category, module, type, severity
POST /api/ai/semantic-search	ai.py	Dense vector semantic defect search
POST /api/ai/fix-code	ai.py	Code Doctor direct syntax & semantic repair
GET /api/issues/{id}/resolution-assistance	issues.py	Generates checklist, similar bugs & fix